



COURS: LANGAGE C (PARTIE_2)

Par: Dr. Mohamed Lemine Sidi



CONTENU DU COURS (PARTIE_2)

- 5. Les Boucles
- 6. Les Tableaux
- 7. Les Pointeurs





5. LES BOUCLES

Les boucles permettent de répéter un ensemble d'instructions plusieurs fois sous certaines conditions. Il en existe principalement trois types:

5.1. La boucle **for**

La boucle for est généralement utilisée lorsque le nombre d'itérations est connu à l'avance. Elle se compose de trois parties :

- **L'initialisation** : elle est exécutée une seule fois au début de la boucle.
- **La condition**: elle est vérifiée avant chaque itération (si elle est vraie, la boucle continue; sinon la boucle s'arrête).
- **L'incrément/décément**: elle est exécutée à la fin de chaque itération.

La Syntaxe:

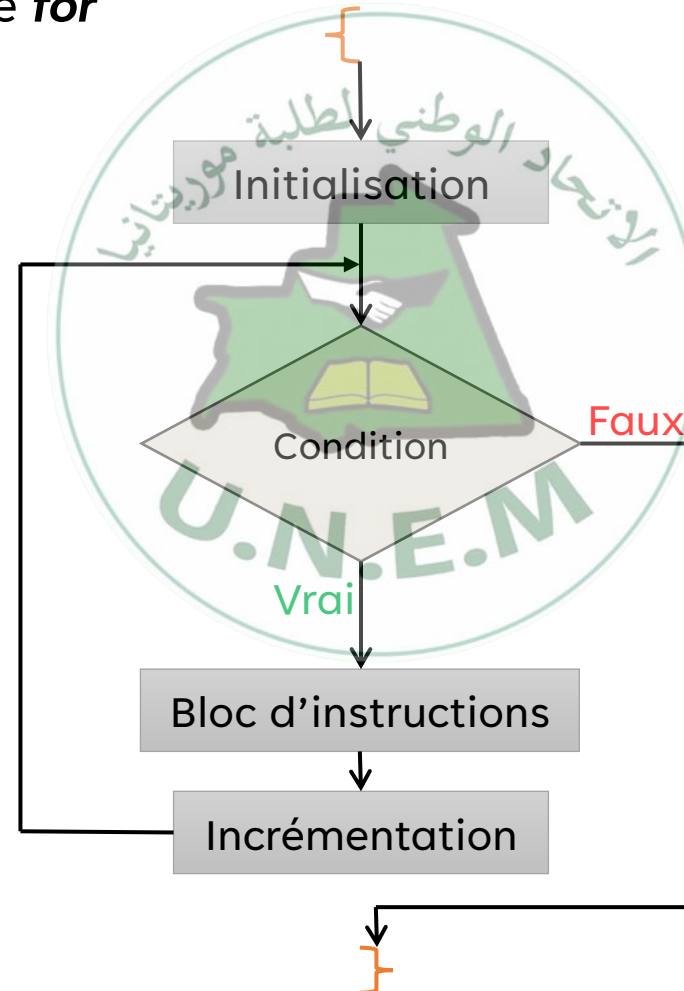
```
for (initialisation; condition; incrément/décément) {  
    // Bloc d'instructions à répéter  
}
```

NB: Une **itération** dans une boucle fait référence à une seule exécution au bloc de code à l'intérieur de la boucle.



5. LES BOUCLES

Le fonctionnement de la boucle **for**





5. LES BOUCLES

Exemple: Une boucle **for** qui affiche les valeurs de *i*, de 1 à 5.

```
#include <stdio.h>
main() {
    int i, N=5;
    for (i = 1; i <= N; i++) {
        printf("i = %d\n", i);
    }
}
```

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

```
-----
Process exited after 0.09912 seconds with return value 6
Press any key to continue . . .
```



5. LES BOUCLES

5.2. La boucle **while**

La boucle **while** est généralement utilisée lorsque le nombre d'itérations n'est pas connu à l'avance, mais que l'on veut répéter une ou plusieurs instructions **tant qu'une condition est vraie**. Elle se compose de :

- **La condition:** elle est testée avant chaque itération. La boucle **while** continue tant que la condition est vraie.

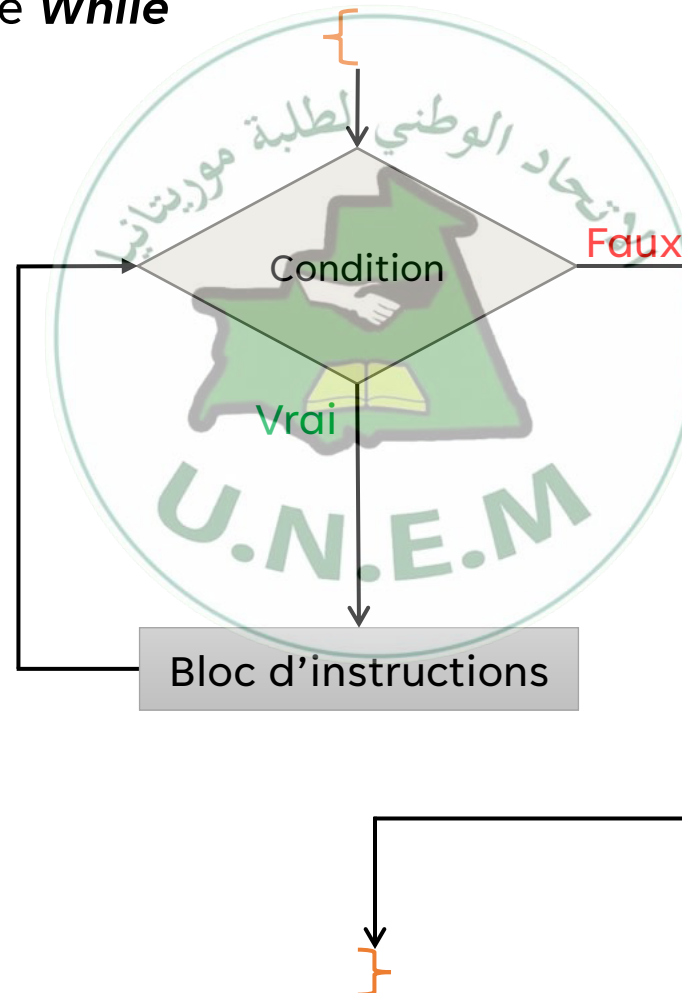
La Syntaxe:

```
while (condition) {  
    // Bloc d'instructions à répéter  
}
```



5. LES BOUCLES

Le Fonctionnement de la boucle **While**





5. LES BOUCLES

Exemple: Une boucle **while** qui affiche les valeurs de **i**, de 0 à 4.

```
#include <stdio.h>
main() {
    int i=0, N=5;
    while (i < N) {
        printf("i = %d\n", i);
        i++;
    }
}
```

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

```
-----
Process exited after 0.09912 seconds with return value 6
Press any key to continue . . .
```




5. LES BOUCLES

5.2. La boucle **do-while**

La boucle **do-while** est similaire à la boucle **while**, mais elle garantit que le bloc d'instructions sera exécuté au moins une fois.

- **La condition:** elle est testée après chaque itération. La boucle **do-while** continue tant que la condition est vraie, et s'arrête si la condition est fausse.

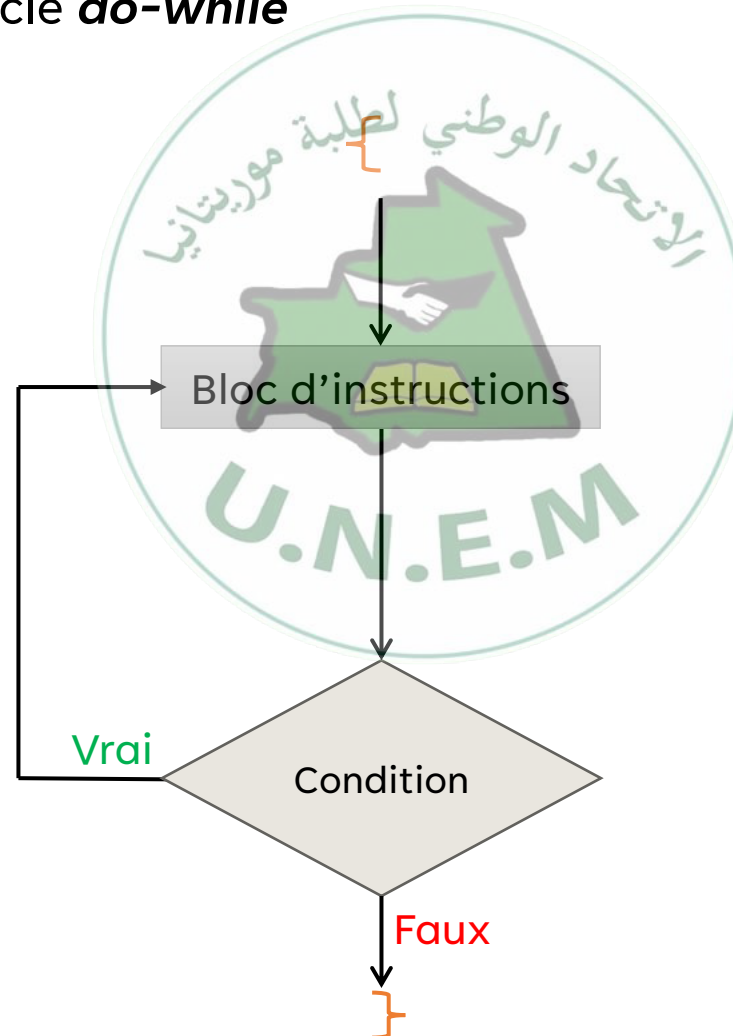
La Syntaxe:

```
do {  
    // Bloc d'instructions à répéter  
} while (condition);
```



5. LES BOUCLES

Le fonctionnement de la boucle **do-while**





5. LES BOUCLES

Exemple: Une boucle **do-while** qui affiche les valeurs de **i**, de 1 à 5.

```
#include <stdio.h>
main() {
    int i=0, N=5;
    while (i < N) {
        printf("i = %d\n", i);
        i++;
    }
}
```

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

```
-----
Process exited after 0.09912 seconds with return value 6
Press any key to continue . . .
```



6. LES TABLEAUX

6.1. Définition

Les tableaux (ou *arrays* en anglais) sont des structures de données permettant de stocker plusieurs éléments **de même type** dans une **variable unique**.

6.2. Déclaration d'un tableau

La syntaxe de déclaration d'un tableau en C est la suivante :

```
type nom_du_tableau[taille];
```

- **type** : Le type des éléments que le tableau va contenir (par exemple, int, float, char, etc.).
- **taille** : Le nombre d'éléments que le tableau peut contenir (doit être une constante entière positive).

Par exemple :

```
int tab[5]; // Un tableau d'entiers de taille 5
float prix[10]; // Un tableau de flottants de taille 10
char lettres[3]; // Un tableau de caractères de taille 3
```



6. LES TABLEAUX

6.3. Initialisation d'un tableau: Il y a plusieurs façons d'initialisation d'un tableau.

a) Initialisation explicite : Donner la taille du tableau et les valeurs pour chaque élément du tableau lors de la déclaration.

Par exemple: `int nombres[5] = {10, 20, 30, 40, 50};`

b) Initialisation sans taille : Donner les valeurs pour chaque élément du tableau lors de la déclaration, sans la précision de la taille du tableau.

Par exemple: `int nombres[] = {10, 20, 30, 40, 50}; // Le compilateur attribue la taille a 5.`

c) Initialisation partielle : Donner la taille du tableau et une partie des valeurs des éléments du tableau lors de la déclaration.

Par exemple: `int tab1[10] = {0}; // Tous les 10 éléments du tableau ont la valeur 0.
int tab2[10] = {1, 2}; // tab2[0]=1, tab2[1]=2, le reste = 0`

d) Initialisation avec boucle : L'initialisation est effectuée après la déclaration via une boucle.

Par exemple: `int tab[10];
for (int i = 0; i < 10; i++){
 tab[i] = i * 2;
}`



6. LES TABLEAUX

6.4. Accès aux Éléments

Chaque élément est accessible via son indice. Et les indices d'un tableau sont de **0** à **taille-1**.

Par exemple:

```
int tab[5] = {10, 20, 30, 40, 50};  
printf("%d\n", tab[0]); // Affiche 10. L'indice=0, c'est le premier élément du tab.  
tab[2] = 100;           // Modifie la valeur le 3ème élément.  
printf("%d\n", tab[4]); // Affiche 50. c'est la valeur du dernier élément du tab.
```

Graphiquement:



6.5. Obtenir la taille d'un tableau

Exemple : Obtenir la taille du tableau de l'exemple ci-dessus.

```
size_t taille_du_tab_en_octets = sizeof(tab); // 5 * sizeof(int)  
size_t nb_elements = sizeof(tab) / sizeof(tab[0]);  
printf("\nNombre d'élément du tableau tab est:%d", nb_elements); // Affiche 5.
```



6. LES TABLEAUX

6.6. Tableau de caractères

En C, un tableau de caractères est souvent utilisé pour représenter des chaînes de caractères. Les chaînes de caractères sont terminées par un caractère spécial '\0' (caractère nul).

Exemple d'utilisation:

```
// Initialisation 1:  
char str[8] = {'B', 'o', 'n', 'j', 'o', 'u', 'r', '\0'};  
// Initialisation 2: il est plus simple d'initialiser une chaîne comme suit :  
char str[]="Bonjour"; // Le compilateur ajoute automatiquement le '\0' à la fin.  
// Affichage :  
printf("%s",str); // Affiche la chaîne: Bonjour.
```



6. LES TABLEAUX

6.7. Les tableaux multidimensionnels

En C, vous pouvez créer des tableaux à plusieurs dimensions. Les tableaux à deux dimensions sont les plus courants et sont souvent utilisés pour représenter des matrices.

a) Tableau 2D (matrice): C'est le tableau à deux dimensions ou matrice. On accède aux éléments en spécifiant deux indices, l'un pour la ligne et l'autre pour la colonne.

Exemple d'une matrice 3x3:

```
int matrice[3][3] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
printf("%d",matrice[1][2]); //Affiche 6. C'est l'element à la ligne 2 et la colonne 3.
```




6. LES TABLEAUX

b) Parcours d'une matrice :

Par exemple: parcourir la matrice de l'exemple précédent.

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        printf("%d ", matrice[i][j]);  
    }  
    printf("\n");  
}
```



7. LES POINTEURS

7.1. Définition

- Les pointeurs sont une fonctionnalité puissante et essentielle du langage C.
- Ils permettent une manipulation directe de la mémoire et de référencer des adresses mémoires spécifiques, offrant une flexibilité et une efficacité accrues.
- Ils nécessitent une gestion attentive, car une mauvaise gestion des pointeurs peut entraîner des erreurs difficiles à détecter, comme des fuites de mémoire (ex. libération incorrecte) ou des violations de mémoire (segfaults. Ex. accéder à une zone de mémoire qui n'est pas allouée au pointeur).

7.2. Qu'est-ce qu'un pointeur

- Un pointeur est une variable qui **stocke l'adresse mémoire** d'une autre variable. Au lieu de contenir une valeur directe, un pointeur **détient l'emplacement en mémoire** où cette valeur est stockée.
- Chaque variable en C a une adresse mémoire unique, et les pointeurs nous permettent d'accéder et de manipuler ces adresses.



7. LES POINTEURS

7.3. Déclaration d'un pointeur

- La syntaxe pour déclarer un pointeur est : `type *nom_du_pointeur;`
 - type:** est le type de données de la variable pointée.
 - L'astérisque *: indique que la variable est un pointeur.

Exemple: `int *ptr1; // pointeur vers un entier`
`char *ptr2; // pointeur vers un caractère`
`float *ptr3; // pointeur vers un nombre à virgule flottante`

7.4. Les Opérateurs de Pointeurs

a) L'opérateur d'adresse (&) : Il renvoie l'adresse mémoire d'une variable. Donc cet opérateur qui est utilisé pour affecter l'adresse d'une variable à un pointeur.

Exemple: `int x = 10;`
`int *ptr = &x; // ptr contient maintenant l'adresse de la variable x.`

b) Opérateur de déréférencement (*) : Permet d'accéder à la valeur située à l'adresse stockée dans le pointeur.

Exemple: `int y = *ptr; // y vaut maintenant 10 (valeur de x)`



7. LES POINTEURS

Explication via un graphique de l'exemple précédent:

```
int x = 10;  
int *ptr = &x; // ptr = 0x306 (pointe sur l'adresse de x).  
int y = *ptr; // y = 10 (valeur de la variable sur lequel ptr pointe)
```

Mémoire		
Adresse	Contenue	Nom de variable
0x127		
....		
0x306	10	x
0x307	#306	ptr
....		
0x577	10	y



7. LES POINTEURS

7.5. Modification de la valeur d'une variable via un pointeur

Les pointeurs permettent de modifier directement les variables auxquelles ils pointent. En utilisant le déréférencement (*), on peut changer la valeur de la variable pointée par un pointeur.

Exemple:

```
int a = 10;  
int *p = &a; // p pointe sur l'adresse de a.  
*p = 15; // Change la valeur de a à 15; c'est équivalent à faire: a=15.  
printf("a=%d", a); // Affiche a=15
```

NB: Un pointeur peut être initialisé a **NULL** (ne pointe vers aucune adresse valide).

```
int *ptr = NULL;  
if (ptr == NULL) {  
    printf("Pointeur non initialisé a une adresse!");  
}
```



7. LES POINTEURS

7.6. Pointeurs et tableaux

- Les tableaux et les pointeurs sont étroitement liés en C. En fait, le nom d'un tableau est un pointeur vers son premier élément.
- Cela signifie que vous pouvez manipuler les éléments d'un tableau en utilisant des pointeurs.

Par exemple:

```
int tab[5] = {1, 2, 3, 4, 5};  
int *p = tab; // p pointe vers le premier élément de tab  
printf("%d", *(p + 2)); // Affiche 3, car p+2 pointe vers tab[2]. Donc *(p + 2)  
est équivalent à tab[2]
```



7. LES POINTEURS

7.7. Pointeurs et allocation dynamique de mémoire

En C, vous pouvez allouer de la mémoire dynamiquement à l'exécution à l'aide des fonctions `malloc()`, `calloc()`, `realloc()` et libérer la mémoire avec `free()` (ces fonctions sont de la bibliothèque `stdlib`). Ces fonctions manipulent des pointeurs pour gérer la mémoire.

Exemple avec la fonction **malloc** :

```
int i;  
int *p = malloc(sizeof(int) * 5); // Alloue de la mémoire pour un tableau de 5  
entiers  
// Initialisation du tableau  
for (i = 0; i < 5; i++) {  
    p[i] = i * 2;  
}  
free(p); // Libère la mémoire allouée
```